Natural-Language Control for Multi-Drone Systems

Overview

This project explores a practical way to operate multiple drones through natural-language instructions while keeping flight execution deterministic, constrained, and auditable.

An operator interacts through either a lightweight Python chat interface or Claude Desktop. The language model interprets the request and selects a typed drone capability exposed through the Model Context Protocol (MCP). Before any command reaches a vehicle, a deterministic safety gate checks the current drone state and the requested action. Approved commands are forwarded to the existing ROS and flight-control stack; rejected commands return a clear reason to the operator.

The model therefore proposes what should happen, while established drone software remains responsible for how the vehicle flies.

Approach

```
Operator prompt
      ↓
Python chat or Claude Desktop
      ↓
Typed MCP tool call
      ↓
Deterministic safety gate
      ↓
Existing ROS skills and behavior trees
      ↓
PX4 / drone
```

The current interface supports:

- reading drone state and telemetry;
- takeoff, landing, and return-to-launch;
- relative movement forward, backward, left, right, up, and down by a requested distance;
- drone yaw and camera-gimbal pitch/yaw control by requested angles;
- image capture and continuous camera capture;
- pointing one drone toward another using their relative positions; and
- starting, monitoring, and stopping existing behavior-tree missions.

The gate checks conditions such as whether the drone is known and connected, whether it is airborne when motion is requested, allowed movement and altitude ranges, battery state, gimbal limits, and the configured operating fence. Every request, gate decision, and result is logged.

First real-world experiment

The first experiment tested the complete system with two physical drones. Commands were issued through the custom Python chat interface and through Claude Desktop using the same MCP tool server.

The demonstration covered takeoff and landing, directional and vertical motion with specified distances, camera pitch and yaw control, image capture, return-to-launch, and a coordinated command in which one drone oriented itself and its camera toward the other drone.

The experiment demonstrated that natural-language requests can be translated into structured, executable drone actions without allowing the language model to directly generate low-level flight commands. It also exercised the safety path: commands that did not match the reported vehicle state could be rejected before reaching the flight stack.

The accompanying video combines the operator interaction with footage of the physical drones responding in the field:

https://drive.google.com/file/d/1_zPsYfUP-ygMsipcDQyDrtAIv74Td3Dw/view?usp=drive_link

Work completed

- Implemented a reusable skill layer over the existing ROS/PX4 drone interfaces.
- Added a deterministic, configurable safety gate and JSONL audit log.
- Added a ROS service that exposes gated skills through a single agent endpoint.
- Added a host-side MCP server with typed schemas for each supported capability.
- Built a lightweight Python chat interface and integrated the same tools with Claude Desktop and Claude Code.
- Added telemetry, camera capture, gimbal control, inter-drone `look_at`, and behavior-tree tools.
- Created a reproducible `soteros-agent` container with rosbridge and the required dependencies.
- Validated the workflow in simulation and with a two-drone real-world experiment.
- Added token, prompt, and end-of-day agent-loop inspection tools for performance analysis.

Current status

The end-to-end architecture is operational in simulation and has been demonstrated on two real drones. The current phase is focused on systematic evaluation, efficiency, and additional safety and verification mechanisms.